

6a. First Come First Serve Process Scheduling

```
#include <stdio.h>
int main() {
    printf("process number :");
    int n;
    scanf("%d",&n);
    int run[n],arr[n],tur[n],wai[n],time=0,p[n];
    float avg_tur=0,avg_wai=0;
    printf("run time :\n");
    for(int i =0;i<n;i++)
    {
        scanf("%d",&run[i]);
    }
    printf("arrival time :\n");
    for(int i =0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n; i++)
    {
        p[i] = i;
    }
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (arr[p[i]] > arr[p[j]])
            {
                int temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    for(int i = 0; i < n ; i++)
    {
        int t = p[i];
        if( time < arr[t] )
            time=arr[t];
        tur[t]=(time+run[t])-arr[t];
        wai[t]=tur[t]-run[t];
        avg_wai += wai[t];
        avg_tur += tur[t];
    }
}
```

```

        time +=run[t];
    }
    printf("run time :\n");
    for(int i =0;i<n;i++)
    {
        printf("Process %d : Turnaround Time: %d Waiting Time: %d \n",
i+1,tur[i],wai[i]);
    }
    printf("\nAverage Waiting Time: %.2f\n", avg_wai / n);
    printf("Average Turnaround Time: %.2f\n", avg_tur / n);
    return 0;
}

```

Breakdown: This code implements FCFS (First-Come, First-Served) scheduling, executing processes in the order they arrive and calculating each process's waiting time (WT) and turnaround time (TAT), as well as the system's average WT and TAT. Being non-preemptive, each process runs to completion once it gets the CPU, making the approach simple and easy to understand, though it can lead to the convoy effect, where short processes wait behind longer ones.

6.b Shortest Job First (Non Pre-emptive version)

```

#include <stdio.h>
int main() {
    printf("process number :");
    int n;
    scanf("%d",&n);
    int run[n],arr[n],tur[n],wai[n],time=0,p1[n],p2[n];
    float avg_tur=0,avg_wai=0;
    printf("run time :\n");
    for(int i =0;i<n;i++)
    {
        scanf("%d",&run[i]);
    }
    printf("arrival time :\n");
    for(int i =0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n; i++)
    {
        p1[i] = i;
    }
}

```

```

for (int i = 0; i < n - 1; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        if (arr[p1[i]] > arr[p1[j]] || (arr[p1[i]] == arr[p1[j]] && run[p1[i]] > run[p1[j]]))
        {
            int temp = p1[i];
            p1[i] = p1[j];
            p1[j] = temp;
        }
    }
}

for (int i = 0; i < n; i++)
{
    p2[i] = i;
}

for (int i = 0; i < n - 1; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        if (run[p2[i]] > run[p2[j]])
        {
            int temp = p2[i];
            p2[i] = p2[j];
            p2[j] = temp;
        }
    }
}

int p3[n];
int completed[n];
for(int i=0;i<n;i++) completed[i]=0;

int count=0, g_index=0;
while(count < n)
{
    int shortest=-1;
    int min_bt=99999;

    for(int i=0;i<n;i++)
    {
        if(arr[i] <= time && completed[i] == 0)
        {
            if(run[i] < min_bt)

```

```

        {
            min_bt = run[i];
            shortest = i;
        }
    }
}
if(shortest == -1)
{
    time++;
    continue;
}
p3[g_index++] = shortest;
wai[shortest] = time - arr[shortest];
time += run[shortest];
tur[shortest] = time - arr[shortest];
completed[shortest] = 1;
count++;
avg_wai += wai[shortest];
avg_tur += tur[shortest];
}
printf("\n");
for(int i =0;i<n;i++)
{
    printf("Process %d : Turnaround Time: %d Waiting Time: %d \n",
i+1,tur[i],wai[i]);
}
printf("\nAverage Waiting Time: %.2f\n", avg_wai / n);
printf("Average Turnaround Time: %.2f\n", avg_tur / n);
return 0;
}

```

Breakdown: This program implements non-preemptive SJF scheduling. It takes CPU burst and arrival times, orders processes by arrival (and burst time for ties), and executes them in shortest-job-first order. It calculates each process's waiting time and turnaround time, tracks execution order in p3[], and computes the average waiting and turnaround times, helping minimize overall waiting time.

6b. Shortest Job First (Pre-emptive version):

```

#include <stdio.h>
int main() {
    printf("process number :");
    int n;
    scanf("%d",&n);

```

```

int run[n],arr[n],tur[n],wai[n],remaining[n],
completed[n],time=0,completed_count=0;
float avg_tur=0,avg_wai=0;
printf("run time :\n");
for(int i =0;i<n;i++)
{
    scanf("%d",&run[i]);
}
printf("arrival time :\n");
for(int i =0;i<n;i++)
{
    scanf("%d",&arr[i]);
}
for(int i=0; i<n; i++) {
    remaining[i] = run[i];
    completed[i] = 0;
    wai[i] = 0;
}
while(completed_count < n) {
    int shortest = -1;
    int min_rem = 99999;
    for(int i=0;i<n;i++) {
        if(arr[i] <= time && completed[i] == 0 && remaining[i] < min_rem) {
            min_rem = remaining[i];
            shortest = i;
        }
    }
    if(shortest == -1) {
        time++;
        continue;
    }
    remaining[shortest]--;
    time++;
    if(remaining[shortest] == 0) {
        completed[shortest] = 1;
        completed_count++;
        tur[shortest] = time - arr[shortest];
        wai[shortest] = tur[shortest] - run[shortest];
        avg_wai += wai[shortest];
        avg_tur += tur[shortest];
    }
}
printf("\n");
for(int i =0;i<n;i++)

```

```

{
    printf("Process %d : Turnaround Time: %d Waiting Time: %d \n",
i+1,tur[i],wai[i]);
}
printf("\nAverage Waiting Time: %.2f\n", avg_wai / n);
printf("Average Turnaround Time: %.2f\n", avg_tur / n);
return 0;
}

```

Breakdown: This program implements Preemptive SJF (SRTF). It takes process count, burst times, and arrival times, then always runs the process with the shortest remaining time among arrived processes. It calculates waiting and turnaround times for each process, shows the averages, and preempts a process if a shorter job arrives.